

RESEARCH ARTICLE

WILEY

Enabling continuous deployment techniques for quantum services

Javier Romero-Álvarez¹ | Jaime Alvarado-Valiente² | Enrique Moguel³ |
Jose Garcia-Alonso⁴ | Juan M. Murillo⁵

Quercus Software Engineering Group,
Universidad de Extremadura, Cáceres,
Spain

Correspondence

Javier Romero-Álvarez, Universidad de
Extremadura, Avda. de la Universidad
s.n., 10003 Cáceres, Spain.
Email: jromero@unex.es

Funding information

MCIN/AEI/10.13039/501100011033 and
by FSE+, Grant/Award Number:
PRE2022-102070; ERDF; European
Commission through the Digital Europe
Program, co-funded by the Spanish
Ministry of Industry, Trade and Tourism,
Grant/Award Numbers: 101083667,
TECH4E-Tech4efficiencyEDIH; Centro para
el Desarrollo Tecnológico Industrial,
Grant/Award Number:
EXP00135977/MIG-20201059; Spanish
Ministry of Science, Innovation and
Universities, Grant/Award Numbers:
PID2021-1240454OB-C31,
TED2021-130913B-I00,
PDC2022-133465-I00; Regional Ministry
of Economy, Science and Digital Agenda of
the Regional Government of Extremadura,
Grant/Award Number: GR21133

Abstract

Early advances in quantum computing have provided new opportunities to tackle intricate problems in diverse areas such as cryptography, optimization, and simulation. However, current methodologies employed in quantum computing often require, among other things, a broad understanding of quantum hardware and low-level programming languages, posing challenges to software developers in effectively creating and implementing quantum services. This study advocates the adoption of software engineering principles in quantum computing, thereby establishing a higher level of hardware abstraction that allows developers to focus on application development. With this proposal, developers can design and deploy quantum services with less effort, which is similar to the facilitation provided by service-oriented computing for the development of conventional software services. This study introduces a continuous deployment strategy adapted to the development of quantum services that covers the creation and deployment of such services. For this purpose, an extension of the OpenAPI specification is proposed, which allows the generation of services that implement quantum algorithms. The proposal was validated through the creation of an application programming interface with diverse quantum algorithm implementations and evaluated through a survey of various developers and students who were introduced to the tool with positive results.

KEYWORDS

continuous deployment, OpenAPI, quantum computing, quantum services, quantum software engineering

Abbreviations: API, application programming interface; AWS, Amazon web services; BPMN, business process model and notation; CD, continuous deployment; CI, continuous integration; DevOps, Development and Operations; PaaS, Platform-as-a-Service; QAOA, quantum approximate optimization algorithm; QFT, quantum Fourier transform; QIR, quantum intermediate representation; RSA, Rivest–Shamir–Adleman; SDK, software-development kit; SOC, service-oriented computing; TOSCA, topology and orchestration specification for cloud applications; TSP, travelling salesman problem.

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial](https://creativecommons.org/licenses/by-nc/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited and is not used for commercial purposes.

© 2024 The Authors. *Software: Practice and Experience* published by John Wiley & Sons Ltd.

1 | INTRODUCTION

Quantum computing is a novel computing technique. It replaces classical bits with quantum bits, or qubits, enabling multiple simultaneous states for each bit and entanglement between bits. These improvements offer increased computational and data processing capacity, which has generated growing interest in harnessing its potential for various applications.¹

Currently, cloud service providers use quantum computers. Nevertheless, certain factors differentiate their use in the cloud from the conventional utilization of classical computers.² On the one hand, quantum computers lack intricate operating systems capable of facilitating concurrent thread execution, as observed in classical systems. Consequently, when employing a quantum computer, task submissions are queued. After fulfilling the request and executing the job, specific procedures must be followed to obtain the desired outcome. These procedures are defined by each provider, and may vary depending on the specific quantum service used.³

However, integrating quantum and classical services is challenging because of the various restrictions imposed by quantum systems. One such restriction is the superposition of states, in which quantum particles can exist in multiple states simultaneously. This superposition is a fundamental aspect of quantum computing but poses challenges when integrated with classical services, which typically rely on deterministic states. In addition, the measurement of the quantum states is probabilistic and introduces uncertainty into the integration process.⁴

To overcome these challenges while incorporating quantum services into existing systems, it is important to acknowledge their limitations. Therefore, quantum software engineering plays a critical role in the development and maintenance of quantum software systems by designing and implementing software solutions that effectively leverage the unique properties of quantum systems.⁵

However, quantum software engineering still has a long way to go to reach the level of classical software engineering. Challenges include hardware disparities and availability and the need to use different programming languages for various quantum machine providers, specific libraries, and other details that depend on each provider.⁵ Ongoing efforts are aimed at bringing quantum computing in the cloud to the same standard as classical computing.⁶ For example, Amazon offers a unified software-development kit (SDK) that enables developers to build quantum algorithms using a single programming language. It also allows for algorithm testing on different simulators and execution on quantum computers from various providers, thereby abstracting hardware differences. Other initiatives have focused on mitigating hardware disparities and availability issues by providing methods for analyzing and optimizing quantum algorithms. These methods estimated the most stable execution by considering a combination of hardware and quantum compilers.⁷

Despite this progress, there is still work to be done in service-oriented computing (SOC) within quantum computing as it becomes more relevant, especially because of its association with cloud computing.⁸ However, significant gaps remain between the understanding and application of the SOC in classical computing and its application in the quantum domain. Currently, some endeavors are focused on bridging this gap by leveraging the advantages of the SOC within the realm of quantum computing. Kumara et al.⁹ proposed an SOC-based methodology that offers a theoretical framework that enables both quantum and classical developers and programmers to collaboratively create hybrid applications. However, the scarcity of resources hampers the ability of developers to develop and deploy quantum services efficiently and extensively.

In the work presented in this study, we focus on developing quantum services that are similar to classical cloud services by adapting existing tools and methodologies to support quantum services.

Therefore, we propose a pipeline for the generation and deployment of quantum services by adapting techniques from the Development and Operations (DevOps) methodology for continuous software integration. In quantum computing, the application of the DevOps process ensures the efficient development, testing, and seamless deployment of quantum services, addressing the needs of both developers and operations teams. Specifically, we propose a modification of the OpenAPI Specification and its code generator to generate quantum services, as well as automation of the continuous deployment (CD) process for their deployment in ready-to-consume containers. This was achieved through a DevOps-based workflow for continuous software integration and deployment using the GitHub Actions* tool. To validate this workflow, a complete process was developed through the following steps: an Application Programming Interface (API) with diverse quantum algorithms implementations, has been specified for the services, the code for these services has been automatically generated with the OpenAPI extension, the services have been automatically deployed on an

*<https://docs.github.com/en/actions>.

Amazon web services (AWS) server using the GitHub Actions tool and Docker,[†] and their correct functioning has been manually verified by analyzing the generated code and making the necessary calls to the services. In addition, to obtain a comprehensive evaluation of our approach, we conducted a survey involving developers and students who utilized the pipeline and its associated tools.

The remainder of this paper is organized as follows. Section 2 presents the background of the present study and discusses the most relevant related studies. Section 3 presents the proposed pipeline for quantum service generation and deployment and details the proposed process for the creation of quantum services. Section 4 demonstrates the feasibility of the generated services and deployment processes using several quantum algorithms, and evaluates the proposed method through a survey of developers and students. Finally, Section 5 presents the conclusions of this study.

2 | BACKGROUND AND RELATED WORK

In this section, we provide an overview of the key concepts and relevant background focusing on the SOC and DevOps methodology that, along with the tools mentioned in the introduction, should be understood by readers so that they can replicate the development of the work.

2.1 | Quantum service-oriented computing

According to current trends, it is expected that complex quantum software systems will adopt a hybrid model that incorporates both quantum components and algorithms, along with classical software elements. A well-established approach for accommodating diverse components is the utilization of SOC.¹⁰ Extensive efforts are already underway in both the industry and research communities to employ SOC for the development of hybrid software systems.¹¹ The industry focuses on offering access to quantum computers through a Platform-as-a-Service (PaaS) framework, whereas the research community concentrates on leveraging SOC for hybrid software system development.⁹

However, creating and operating quantum-service-oriented software remains a complex undertaking that differs significantly from the development of classical services to which professionals are accustomed. The absence of advanced operating systems hinders the deployment of quantum services in the same manner as classical services. Alternatively, a classical service can be deployed to execute a quantum task upon invocation, thereby introducing an additional layer of indirection and complexity into the system. Nonetheless, the limited abstraction level of most quantum programming languages and algorithms leads to heavy reliance on the specific hardware for which the quantum source code is designed, thereby impeding developers from leveraging the availability of different quantum computers via cloud platforms.¹²

The limited availability of advanced tools and methodologies for quantum service development has led to the utilization of low-abstraction and error-prone techniques by developers, which lack the benefits and advantages offered by modern software development tools. Various research initiatives are emerging to address these challenges, in addition to the need to know the specific language and particularities of each quantum provider.

Weder et al.¹³ conducted a study focusing on the issues associated with orchestrating hybrid systems that involve both quantum and classical services. To address these concerns, they proposed the adoption of an orchestration mechanism based on the topology and orchestration specification for cloud applications (TOSCA) to coordinate different services.

A similar study by Garcia-Alonso et al.¹⁴ introduced the concept of a Quantum API Gateway that dynamically determines the most suitable quantum computer to execute a specific quantum service. They also explored the key problems of quantum services¹⁵ and presented a deployment guide.¹⁶

Furthermore, emerging endeavors in hybrid quantum-classical computing aim to provide APIs. Examples of such efforts include qiskit runtime¹⁷ and quantum intermediate representation (QIR).¹⁸ These initiatives enable users to execute workloads efficiently, while serving as standardized interfaces between different programming languages and quantum computing platforms.

Although these proposals elevate the level of abstraction in quantum service development and simplify the creation of intricate hybrid solutions in which classical and quantum services coexist, they still fall short of the classical development of service-oriented software. To address this limitation, additional support is required to facilitate the development of

[†]<https://www.docker.com/>.

quantum services. This support enables current service developers to transition more easily into the quantum domain, thereby mitigating the shortage of a skilled quantum workforce and fostering the development of a new generation of hybrid systems.¹⁹

2.2 | Leveraging continuous deployment for quantum software

DevOps is a software development methodology that seeks to integrate and synchronize the efforts of the development and operation teams to optimize the software delivery process.²⁰ It places particular emphasis on fostering collaboration, implementing automation, and enhancing communication channels to improve the velocity, quality, and dependability of software deployment.

In the field of quantum software development, DevOps has assumed significant relevance by addressing the distinctive challenges inherent in quantum computing. For instance, owing to quantum noise-induced errors, quantum software requires stricter testing and validation protocols than classical software. DevOps can facilitate the automation of this testing and validation procedures, thereby enhancing the dependability of the quantum software.²¹ Furthermore, quantum software exhibits heightened complexity and computational demands compared with classical software. DevOps can alleviate this complexity by automating the deployment, scaling, and maintenance of quantum software, thereby augmenting its efficiency and scalability throughout its development lifecycle.²²

Continuous integration (CI) and continuous deployment (CD) represent fundamental practices within the DevOps methodology that seek to automate the software delivery process. CI involves the integration of code modifications into a shared repository, whereas CD involves the prompt automated deployment of software updates to the production environment upon completion.²³

Currently, a few approaches exist for integrating quantum algorithms into the various developmental stages of the DevOps cycle. An example of applying continuous deployment principles can be found in the work of Gheorghe-Pop et al.,²¹ who proposed an adaptation of the traditional DevOps process to the quantum realm, termed Quantum DevOps. This study makes a significant contribution by introducing and advocating the concept of Quantum DevOps, emphasizing its potential benefits and importance.

These proposals account for quantum computing characteristics such as qubit quantity, cloud accessibility, and error rates of individual quantum machines. However, no tools or implementations have been developed to aid developers. Despite proposals to incorporate quantum algorithms into CD stages, certain aspects of quantum CD remain unaddressed by these aforementioned proposals. In particular, the critical phases of these cycles, such as building and deployment, require consideration. Building and deployment are crucial stages involving the creation of software packages and their subsequent deployment in the production environment. This study aims to encompass these aspects and optimize the generation and deployment processes, thereby expediting the delivery of quantum-computing applications.

3 | PIPELINE FOR QUANTUM SERVICES GENERATION AND DEPLOYMENT

This section comprehensively explains the proposed pipeline for generating and deploying quantum services using OpenAPI, Docker, and GitHub. OpenAPI, which has been modified to support quantum services, offers a standardized specification for the development and generation of APIs, whereas Docker facilitates the creation and deployment of containers capable of deploying these APIs. By combining these technologies, we propose a flexible and robust approach for the creation, management, and deployment of quantum services.

The complete pipeline uses GitHub Actions to automate the process, thereby enabling CD of quantum services. GitHub Actions are a feature provided by GitHub that allow developers to automate various tasks and workflows within their software development processes. It acts as a continuous integration and deployment (CI/CD) tool, which helps automate the building, testing, and deployment of code changes. In this study, GitHub Actions were used to automate code generation and deployment for quantum services. It was integrated with the modified OpenAPI specifications to facilitate a seamless transition from manual processes to automated workflows.

The proposed CD pipeline is illustrated as a BPMN diagram in Figure 1 as a Business Process Model and Notation (BPMN) diagram.

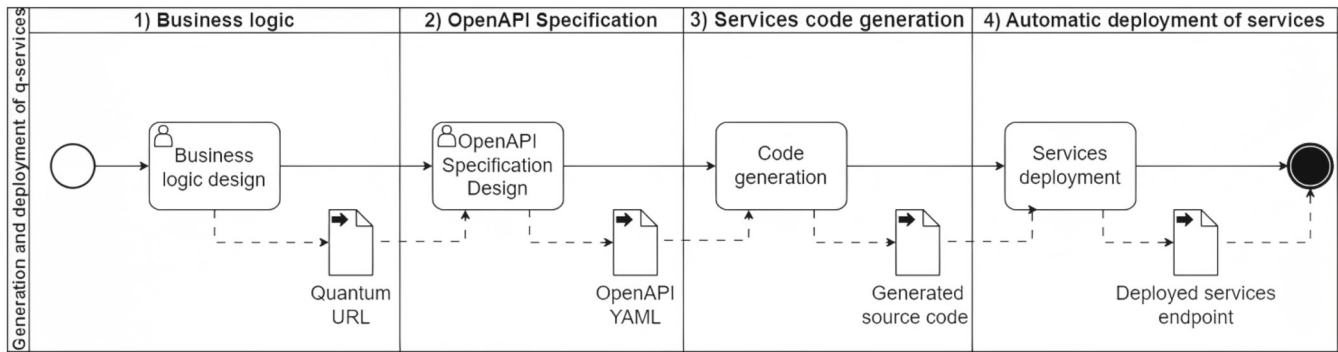


FIGURE 1 CD pipeline for quantum services generation and deployment.

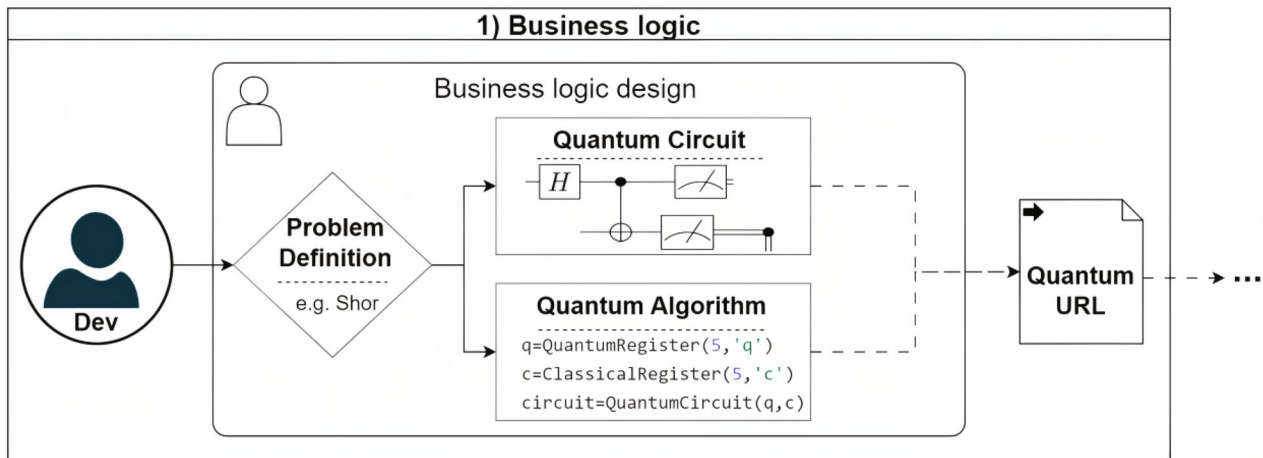


FIGURE 2 Definition of the business logic.

To facilitate the replicability of this study, the complete replication package is publicly available[‡]. It includes all essential components and configurations necessary to enable the seamless deployment of quantum services.

In the following sections, we discuss the four main phases of the proposed pipeline (Figure 1) and its advantages over traditional methods. In Section 3.1, we describe defining the business logic of quantum services as circuits. Section 3.2 expounds how these circuits, with the OpenAPI specifications, enable the definition of services. Section 3.3 explains the subsequent generation of service code based on the aforementioned specification. Finally, Section 3.4 outlines the automatic deployment of these quantum services.

All steps of this process are explained, taking as an example a service that implements Shor's algorithm, one of the best-known algorithms in quantum computing.

3.1 | Business logic

The initial phase of the pipeline implies a definition of the business logic for quantum services as a quantum circuit, as shown in Figure 2.

To seamlessly integrate the business logic of quantum services, we leverage a graphical quantum programming tool that facilitates the construction of quantum circuits through intuitive drag-and-drop operations. Specifically, the

[‡]<https://doi.org/10.5281/zenodo.10477708>.

web-based quantum programming tool Quirk.[§] Developed as an open-source software solution utilizing the JavaScript programming language, Quirk streamlines the rapid design of quantum circuits. Moreover, it provides programmatic access to the composed quantum circuits via its graphical editor.

Note that Quirk can be substituted with any other quantum circuit creation tool that offers programmatic access to the underlying code, thereby ensuring compatibility with the OpenAPI specifications.

Furthermore, as an alternative to employing a graphical circuit editor, we offer the option of directly incorporating quantum circuits into the service endpoint through the URL of the repository hosting the Python file with the source code of the circuit, such as Qiskit code.

The URL containing the quantum circuit, whether as a circuit constructed in Quirk or an already implemented circuit, serves as the input for the subsequent phase of the pipeline. The circuit implementation of the Shor's algorithm example used is available in the replication package is publicly available, we can unify them.[¶]

3.2 | OpenAPI specification

To successfully implement a classical service within the OpenAPI specification, developers must consider two crucial aspects: the business logic, which encompasses the service's functionality, and the service endpoint, which governs how external clients interact with the service.

The OpenAPI specification plays a pivotal role in defining the service endpoint by employing a language-independent standardized interface tailored for RESTful APIs. This standardized approach allows users to explore and understand a service's capabilities without requiring direct access to the source code or extensive documentation. This enables us to define the service itself, including the specifications of the input and output parameters. By leveraging a source code generator, a code structure is automatically generated based on this specification, which is explained in the next phase of the pipeline. Developers can select their preferred programming language, and subsequently integrate the business logic into the generated code, resulting in a fully functional service.^{24,25} Furthermore, from a scientific standpoint, OpenAPI enables the smooth exchange of information between diverse software systems by establishing a common vocabulary to describe API functionality.

To address the current state-of-the-art in quantum services, we adopted an approach similar to that employed for classical service implementation, utilizing OpenAPI. To achieve this, an extension of the OpenAPI specification was devised, encompassing custom properties tailored to define quantum applications.

Specifically, the generated OpenAPI specification was extended with custom properties to incorporate the quantum aspects of the service, such as the URL obtained in the previous step—Figure 3. This extension allows for the definition of quantum applications using OpenAPI. By integrating the specific properties of quantum computing into the OpenAPI specification, we pave the way for the seamless integration of quantum services into a broader service ecosystem.

In this article, we propose the utilization of custom properties, also known as specification extensions or provider extensions, to extend OpenAPI specifications and enable the inclusion of quantum circuits. These custom properties incorporate supplementary information into the API contract definition, thereby expanding the scope of the specification.

Figure 3 also shows a fragment of the YAML API contract with the service endpoint of Shor's algorithm, showcasing some custom properties and parameters added to the specification. This YAML contract contains custom quantum properties and quantum provider-specific details, and the URL of the circuit is input to the next phase of the pipeline.

Next, the most important properties and parameters are briefly described. The first custom property, called *x-quantumCode*, contains a URL that hosts the implemented circuit code, or Quirk URL. The second custom property, *x-quantumProvider*, allows developers to designate a service provider on which a quantum service will be executed. Presently, it allows the selection between two prominent service providers: IBM Quantum, which provides the Qiskit Development Kit, and Amazon Braket, which offers a suite of quantum computing services. We selected these two providers because they are two of the most widely used today and cover a wide range of languages and hardware vendors.

Additionally, two parameters, namely *machine* and *shots*, were incorporated into the specification to ensure proper execution of the quantum services. By focusing on these key parameters, the OpenAPI Specification remained concise and targeted, providing the flexibility and control for users to interact effectively with quantum services. The decision not to include additional parameters beyond the *machine* and *shots* in the OpenAPI Specification is based on prioritizing

[§]<https://algassert.com/quirk>.

[¶]<https://doi.org/10.5281/zenodo.10477708>.

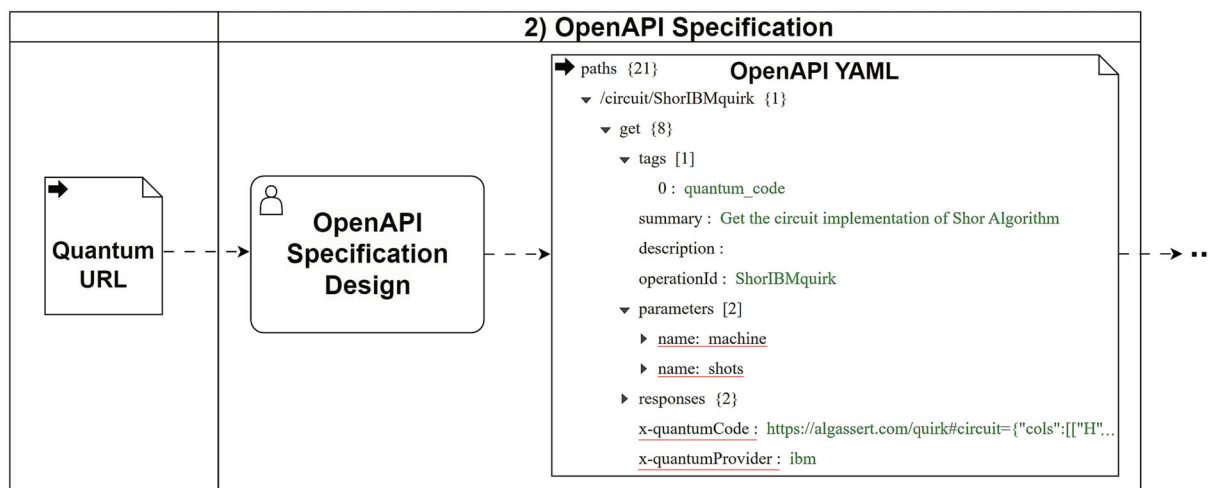


FIGURE 3 OpenAPI specification for quantum services.

the accurate representation of quantum functionality and maintaining compatibility with the standardized approach provided by OpenAPI.

The *machine* parameter allows the service client to select the specific machine on which the quantum algorithm will be executed dynamically from the options offered by the service provider specified in the custom *x-quantumProvider* property. This flexibility is necessary, because Amazon and IBM provide different quantum computers and simulators for developers.

Moreover, the *shots* parameter allows the specification of the desired number of times the circuit should be executed on the chosen quantum machine for each service request. This parameter is essential for controlling the statistical sampling and obtaining reliable results from quantum algorithms.

3.3 | Services code generation

In the third phase of the pipeline, the extension proposed for the OpenAPI Code Generator[#] is employed to automatically generate the source code for quantum services from the YAML specification of the previous step, as shown in Figure 4.

The OpenAPI Code Generator, developed by the OpenAPI initiative, is a web tool that enables the creation of server applications and client APIs based on OpenAPI specifications. Comprising various modules and libraries, this code generator facilitates code generation in multiple programming languages. In this study, we extended this tool by introducing a new library called *python-quantum*, which leverages the existing Python library for API generation, namely Flask,^{||} which is a lightweight and widely adopted Python web application framework. Consequently, the generated source code implements quantum API endpoints in Python using Flask software. These endpoints deploy and execute the designated quantum algorithm on the provider and machines specified by the developer, utilizing custom properties incorporated into the OpenAPI specifications as explained above.

Thus, these endpoints contain the source code to execute a quantum task for the designated service provider, as indicated by the custom property *x-quantumProvider*. The source code of the quantum task was generated for the specified provider by employing the quantum circuit specified by the *x-quantumCode* property.

Depending on the provider with which the service is deployed, it is necessary to generate code in the specific language of the provider. Therefore, a previous translation process is necessary for services where the logic has been defined through Quirk and not Python code.

For this process, we developed a translator API, called *Q-Trans*, to perform translation accurately and efficiently and handle the complexities of quantum programming languages more proficiently. The *Q-Trans* tool features an endpoint

[#]<https://openapi-generator.tech/>.

^{||}<https://flask.palletsprojects.com>.

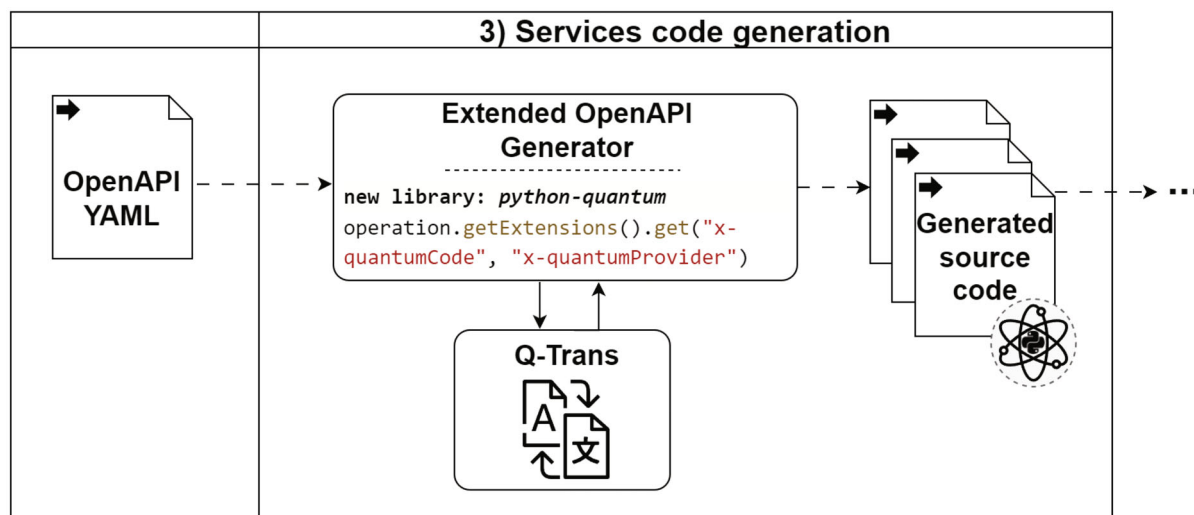


FIGURE 4 Quantum services code generation.

for each supported provider. These endpoints receive the Quirk URL, format it accordingly, and generate code using the specified gates in the respective programming language. The quantum machine on which the algorithm was executed was provided as a parameter by the client.

The generated source code encompasses the essential libraries of the quantum provider, the classical wrapping service, and the supported machines available at the quantum provider. In addition, it incorporates business logic derived from a quantum circuit, which is translated into a language supported by a quantum provider. In the replication package^{**} is available the code of the generated service that encapsulates Shor's algorithm circuit.

The resulting quantum services can be deployed and invoked in the subsequent phase of a pipeline by making RESTful calls to the endpoints that encapsulate the quantum algorithm code.

3.4 | Automatic deployment of services

Once the source code of the quantum service is generated, in the final phase, we focus on automating its deployment. This automation ensures that the services that were automatically generated in the previous phase are subjected to a systematic deployment process facilitated by the CD using software containers, as shown in Figure 5.

Containers are useful solutions for isolating software components individually and configuring multiple deployments. For this purpose, the Docker development tool has been used, an open-source platform that allows the deployment of applications within software containers, also allows the process to be automated.

The tool developed to automate the creation and deployment of containers in this project is referred to as *Q-Deployment*. This tool encapsulates the generated source code in a Docker container and then deploys it. This tool and GitHub Actions are related in the sense that they both play a role in automating the creation and deployment of containers for quantum services.

Thus, the source code generated for the quantum services was encapsulated within a Docker Registry container. This encapsulation process utilizes a pre-defined *Dockerfile* that incorporates the instructions to prepare the container appropriately for deployment.

To enhance the replicability of the system, a foundational registry container has been proposed and made available.^{††} This container was built on a Python image and was equipped with all the requisite dependencies for seamless execution. By leveraging this pre-configured container, the time-consuming process of container creation was significantly expedited.

Finally, the container was deployed through one of the server ports. To facilitate this, the system checks the ports from a pre-defined list and assigns the first available port. It is important to note that in the present context, services can only

^{**}<https://doi.org/10.5281/zenodo.10477708>.

^{††}<https://doi.org/10.5281/zenodo.10477708>.

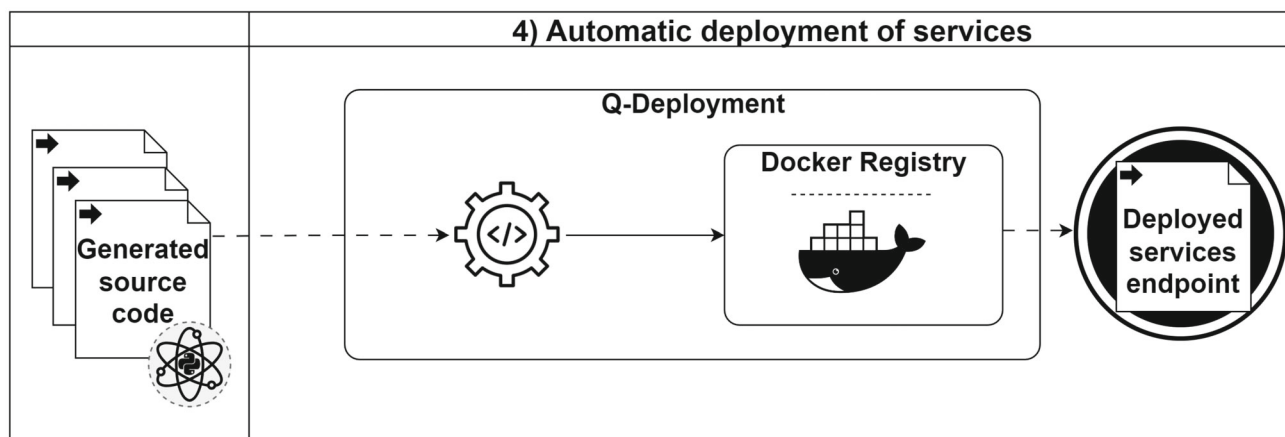


FIGURE 5 Automatic deployment of services.

be deployed on the same server on which the entire system is deployed. Subsequently, the user is furnished with a link to the server appended to a designated port number, providing access to the generated services. In the following link^{††} is deployed the Shor service, from the example used in the previous steps.

Overall, the proposed pipeline demonstrates a systematic and automated approach for the generation and deployment of quantum services within a containerized environment. By leveraging GitHub Actions and the modified OpenAPI specifications, developers can seamlessly transition from manual processes to automated code generation and deployment.

Further details regarding the visualization and execution of the services can be found in Appendix A for a comprehensive description of the service display and deployment procedures.

4 | VALIDATION

In this section, we validate the complete pipeline for the CD of quantum services. Section 4.1 validates the tool by generating and deploying quantum services that contain 40 real quantum algorithms. Section 4.2 evaluates the proposed pipeline by surveying different developers and students who were introduced to the tool.

4.1 | Generating quantum services with quantum real algorithms

In this study, we present a procedural process for the automated generation and deployment of quantum services, focusing specifically on the CD process. The efficacy of this pipeline is evaluated by creating an API encompassing multiple services, each encapsulating diverse implementations of quantum algorithms.

4.1.1 | Implementations of quantum algorithms

To verify the functionality and deployment of the proposed pipeline, we employ a range of gate-based quantum circuit algorithms representing various quantum algorithms. These algorithmic implementations cover a wide range of applications, including both quantum-classical hybrids and purely quantum algorithms.

The quantum algorithms utilized for this validation process are publicly available,^{§§} wherein 20 of them have been implemented in Python code, while the remaining 20 have been implemented using the quantum circuit composer Quirk. Consequently, the API consists of 40 distinct services, as each implementation was generated for both AWS and IBM platforms, denoting the corresponding providers within the OpenAPI specification.

^{††}<http://quantumservicesdeployment.spilab.es:8093/ui/>.

^{§§}<https://doi.org/10.5281/zenodo.10477708>.

TABLE 1 Summary of the used quantum circuits.

Number	Circuit
1	Shor's algorithm
2	Bernstein-Vazirani
3	Grover's algorithm
4	Deutsch-Jozsa Algorithm
5	Simon's algorithm
6	Traveling salesman problem (TSP)
7	Quantum teleportation
8	Quantum phase estimation
9	Quantum fourier transform (QFT)
10	Quantum approximate optimization algorithm (QAOA)

Among the algorithms employed, we highlight the utilization of Grover's algorithm,²⁶ which is a quantum search algorithm renowned for its ability to search an unsorted database of N items in $O(\sqrt{N})$ -time as opposed to the $O(N)$ -time complexity of classical algorithms. This algorithm has applications in tasks, such as database searches and pattern matching.

Another significant algorithm is Shor's algorithm,²⁷ which surpasses the efficiency of classical algorithms in integer factorization. Shor's algorithm demonstrates exponential speedup and is of paramount importance in cryptography, as the security of widely used public-key cryptosystems, such as Rivest–Shamir–Adleman (RSA), relies on the computational infeasibility of factoring large integers.

Additionally, Simon's algorithm,²⁸ which exploits quantum superposition to detect hidden patterns in Oracle outputs, was integrated into the framework, further underscoring its versatility in quantum computing.

To assess the generation and deployment processes, we implemented the quantum approximate optimization algorithm (QAOA).²⁹ This class of algorithms has applications in solving optimization problems within domains such as finance and engineering.

Finally, our collection of algorithmic implementations, which is summarized in Table 1 includes other circuits employed as subroutines in quantum computing.

Through a comprehensive evaluation of these algorithmic implementations, we substantiated the efficacy and viability of our automated approach for the generation and deployment of quantum services. These findings lay a strong foundation for further research and development in this domain, propelling the practical applications of quantum computing.

4.1.2 | Results of the deployments

The proposed approach for automatic generation and deployment of quantum services was evaluated to assess its effectiveness. Several aspects were considered in determining the correctness of the deployment. First, it was essential to verify that the circuit code was accurately encapsulated within the service, including all libraries necessary for its execution. Second, the functionality of the API must be validated to ensure that it functions correctly. Finally, the encapsulated quantum circuits are executed and evaluated for errors, focusing on whether they produce the expected results.

To test the latter, different services were subjected to experiments by running the circuits on multiple quantum machines provided by AWS and IBM. The aim was to assess whether the deployed services operated as intended and could deliver the desired outcomes.

The experiments involved three different methods of generating circuits: two using Qiskit and Amazon Braket and the other involving circuits generated by Quirk Composer, for a total of 40 services. The results revealed that all 20 quantum services utilizing Qiskit and Amazon Braket generated circuits were successfully deployed. However, for services containing circuits generated by Quirk Composer, 17 out of 20 were deployed correctly, whereas three experienced deployment failures—Figure 6.

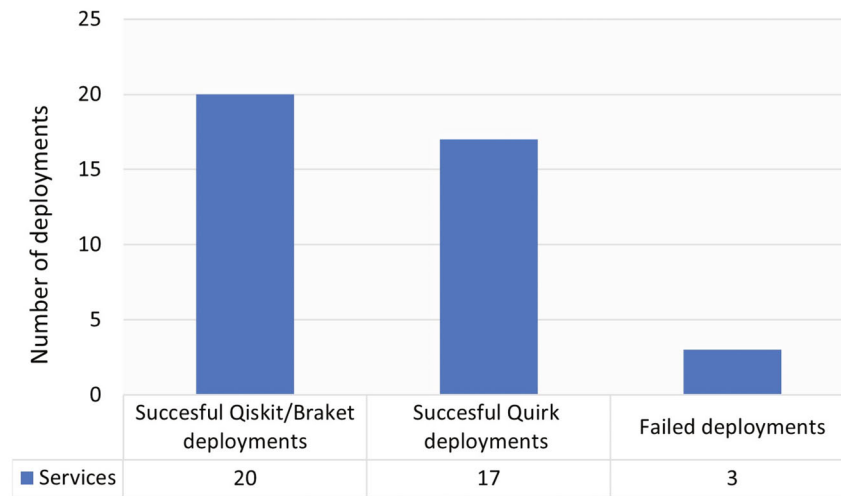


FIGURE 6 Number of failed and successful deployments.

A more thorough analysis revealed that one of the potential causes for failed deployments was the limited support for specific circuit gates, particularly phase gates, within the code generator employed by *Q-Trans*. This limitation prevents the code generator from effectively handling and incorporating all the types of circuit gates present in quantum circuits.

Quirk-specific phase gates, in which an angle can be specified, are not properly translated or included in the generated code, resulting in errors during deployment. For example, phase gates are used in QAOA circuits. Consequently, the affected quantum services that utilize circuits with phase gates encounter deployment failures. To overcome this issue, a new version of the code generator was developed. The aim is to enhance the capabilities and ensure seamless integration and translation of all circuit gates, including phase gates and other gates that have not been contemplated. By addressing this limitation, the new version of the generator is expected to facilitate the successful deployment of quantum services that contain circuits with a broader range of gate types.

Overall, the proposed approach demonstrates a promising success rate for automatic quantum service generation and deployment. These results highlight the potential of the proposed approach for significantly reducing the time and effort required for circuit generation and deployment. Further research can be conducted to identify the factors contributing to deployment failures and explore possible solutions to mitigate them.

4.2 | Evaluation of the automatic quantum deployment pipeline

The proposed pipeline for the Continuous Deployment of quantum services was also evaluated by developers and students. This validation consisted of a training course aimed at helping software engineers address the difficulties encountered when developing quantum services.

A total of 62 people participated in the training course, which was held at two universities—the University of Extremadura^{¶¶} and the University of Málaga^{##}—as well as at the Bertinoro International Spring School^{|||} and as a tutorial at the International Conference on Web Engineering.^{***}

This course introduces the basic concepts of quantum computing and shows the structure of a quantum program, as well as the main techniques and patterns currently used to develop them. Subsequently, the proposed pipeline is used to develop the same programs. At the end of the course, a survey was conducted to determine the usefulness of the tool.

¶¶ <http://qserv.spilab.es/curso-ux-2023-introduccion-al-desarrollo-de-software-cuantico/>.

https://www.enseñanzaspropias.uma.es/informacion_curso.php?id_curso=6903577.

||| <https://tempesta.cs.unibo.it/projects/BISS/2023/courses/#hybrid-quantum-computing>.

*** <https://icwe2023.webengineering.org/quantum-web-services-development-and-deployment/>.

4.2.1 | Problems and questions

As mentioned in the previous sections, several problems related to the development and deployment of quantum services have been detected, such as hardware differences and availability and a lack of tools and methodologies to support the development of quantum services on par with classical cloud services. Therefore, we propose a pipeline for generating and deploying quantum services based on CD.

These issues led us to conduct a survey to collect feedback from developers/students on their experience with existing methods and with this proposed approach. The questions on which the survey was based were as follows:

- **General questions** have been posed to obtain a profile of the survey participants. Such as their employment status, academic background, or their experience in quantum computing.
- **Technical questions** have been posed to analyze the current perception of quantum software development in terms of the aforementioned problems and mainly to corroborate the easiness and usability of the process presented:
 - **RQ1. In your opinion, what is the main aspect of quantum computing that should be improved from a software engineering point of view?** This question was raised to obtain the perspective and opinions of respondents on specific areas for improvement at the intersection of quantum computing and software engineering.
 - **RQ2. Have you designed any quantum circuits and executed them in a quantum machine or simulator before?** This question aims to evaluate the level of knowledge and practical experience of the respondents in quantum computing. Specifically, in the design and implementation of quantum circuits.
 - **RQ3. Have you developed or deployed any quantum services before?** This question is related to the previous one, but in this case, the answers to this question will provide information on the familiarity of the respondents with the development of applications and services in quantum computing.
 - **RQ4. How challenging do you find the process of generating and deploying quantum services using current methods?** This question seeks to assess the degree of complexity and difficulties perceived by respondents in the development and implementation of quantum services using existing methods seen in the course.
 - **RQ5. Is it essential for you to have tools that will allow the development and deployment of quantum services?** This question is useful to know if the respondents see the need for tools that facilitate the development process of quantum services.
 - **RQ6. Have you understood the concepts explained in the course?** This question allows us to know if the contents explained during the course have been understood or not, with a view to future improvements both in the proposed process and in the course.
 - **RQ7. How much easier do you think it would be to develop and deploy quantum services using the semi-automatic method presented compared to the current methods?** This question allows us to evaluate the opinion of the respondents on the improvement degree that the presented semi-automatic approach can offer in terms of simplicity and efficiency in the development and deployment of quantum services.
 - **RQ8. How effective do you think the Continuous Deployment approach would be in simplifying and facilitating the development process of quantum services?** Responses to this question will provide information on the respondents' expectations and perceptions of the potential benefits and advantages of the Continuous Deployment approach compared to existing methods.
 - **RQ9. How important is it for you to have a simplified workflow for generating and deploying quantum services?** This question allows us to determine the importance that respondents assign to having a simplified workflow for generating and deploying quantum services.

Appendix B contains all questions, answers, and obtained results. Analysis of the results in the following subsection highlights the most crucial findings.

4.2.2 | Main results of the survey

With regard to the main results for the general questions, Figure 7A,B shows the percentages of the 62 participants' profiles in the survey. Of these, 51% are computer scientists, 85% are students, and practically all have a few years of experience in quantum computing.

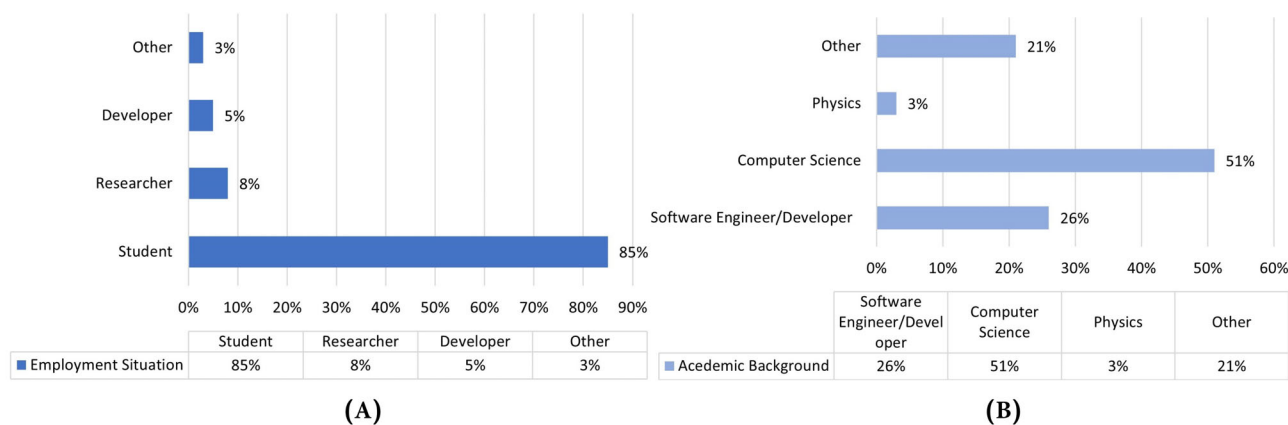


FIGURE 7 Results obtained from participants' profiles. (A) Current employment situation. (B) Academic background.

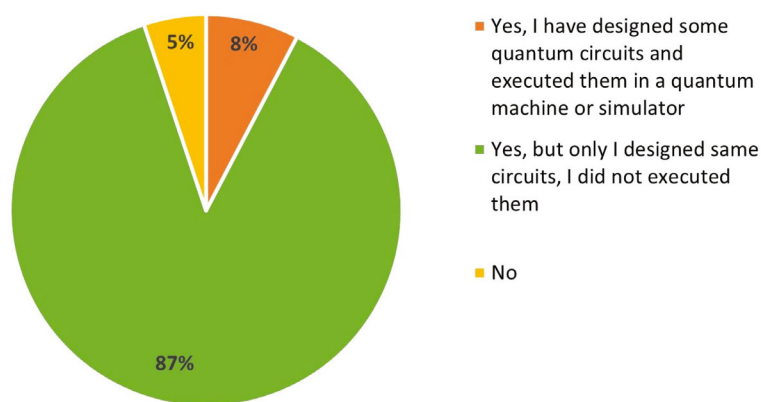


FIGURE 8 Results of RQ2.

Considering the results regarding the research questions related to the current perception of quantum software development, with the responses of RQ1 we can conclude that the main aspect of quantum computing that should be improved from a software engineering perspective is the developer experience. The participants highlighted the need for more accessible programming languages and tools, improved availability and accessibility of quantum machines, standardization of access across vendors, and increased levels of abstraction in quantum computing.

For RQ2—Figure 8—most participants (87%) designed quantum circuits but did not execute them on real machines. This is related to RQ3—Figure 9—where 97% of the participants had not developed or deployed any quantum services before, but had worked in quantum computing.

Related to the lack of tools and methods for the development of quantum services, the responses from RQ4—Figure 10—indicate that generating and deploying quantum services using current methods is challenging by the respondents. 38% find it somewhat challenging, whereas 44% found it very challenging. This indicates the significant difficulties associated with the current methods.

Finally, in RQ5—Figure 11—most of the respondents (almost 80%) considered it essential to have tools that enable the development and deployment of quantum services. This is consistent with the high percentage of respondents who found the current process challenging, indicating a strong demand for improved tools and methods.

In summary, the results of this part of the survey highlighted the importance of improving developers' experience in quantum computing. The participants expressed the need for accessible programming languages, better access to quantum machines, standardization, and increased abstraction. Although most participants have designed quantum circuits, they have not yet deployed quantum services. The current process for generating and deploying quantum services is challenging. These findings emphasize the significance of developing tools and methods to facilitate the development and deployment of quantum services, as expressed by most respondents.

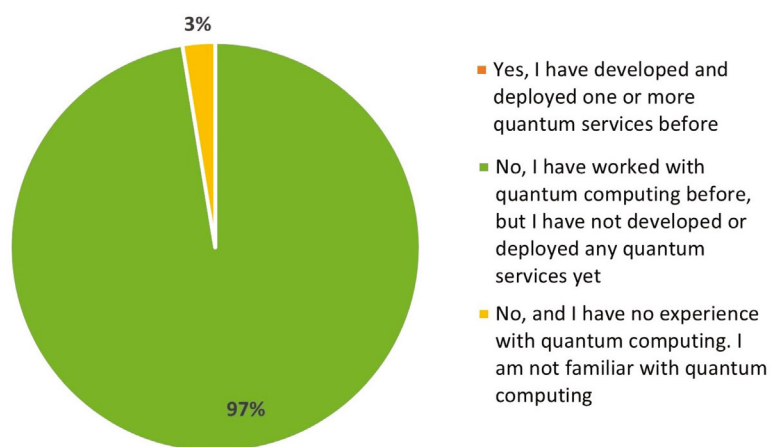


FIGURE 9 Results of RQ3.

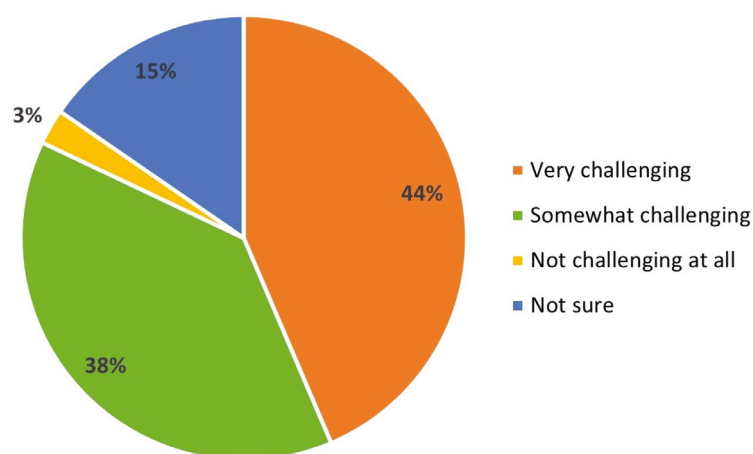


FIGURE 10 Results of RQ4.

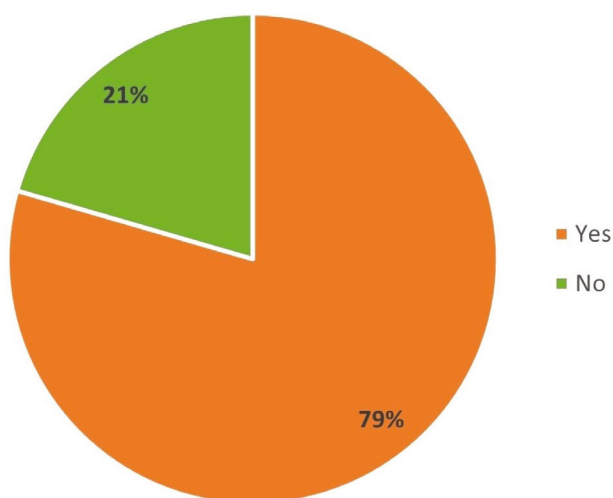


FIGURE 11 Results of RQ5.

Regarding the results related to the questions that specifically evaluated the usefulness of the proposed process, we obtained positive results that validated our proposal.

First, Figure 12 shows the results for RQ6, indicating that most participants (over 97%) clearly understood the concepts explained during the course and learned to use the proposed tool. This validates that our process is easy to use, even for developers with little experience in quantum computing.

Furthermore, regarding the results of question RQ7, as shown in Figure 13, a total of 64% (33% + 31%) of the participants believed that using semiautomatic methods would make the process either “*much easier*” or “*somewhat easier*”. This suggests that a significant proportion of participants perceived semiautomatic methods as beneficial for simplifying the development and deployment of quantum services. In addition, 10% of the respondents were unsure about the impact of semi-automatic methods. This suggests that further research, development, and user feedback are required to assess the true efficacy and feasibility of the proposed semi-automated methods in quantum service development and deployment.

However, RQ8 also validates our proposal for the CD of quantum services—Figure 14—, where most respondents (59%) perceive the CD approach as “*very effective*” in significantly streamlining and facilitating the development process of quantum services. This suggests that the participants recognized the potential benefits of the proposed CD process to improve efficiency, reducing manual effort, and ensuring rapid iterations and updates in the development of quantum services. However, 23% of the participants believed that there may still be challenges or limitations that need to be addressed.

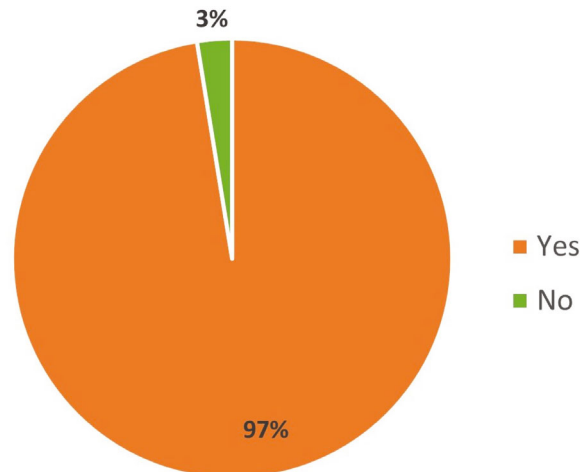


FIGURE 12 Results of RQ6.

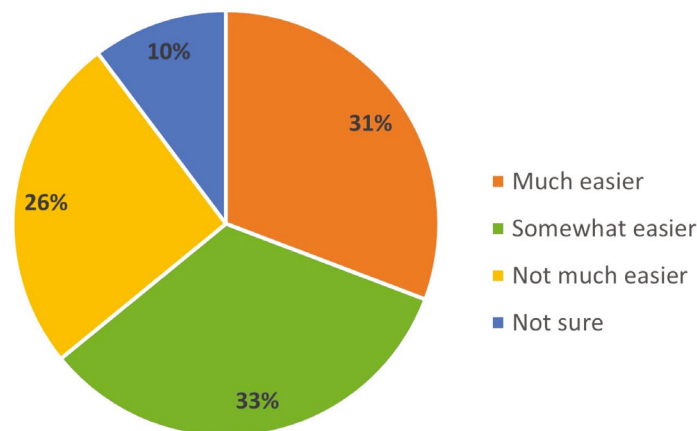


FIGURE 13 Results of RQ7.

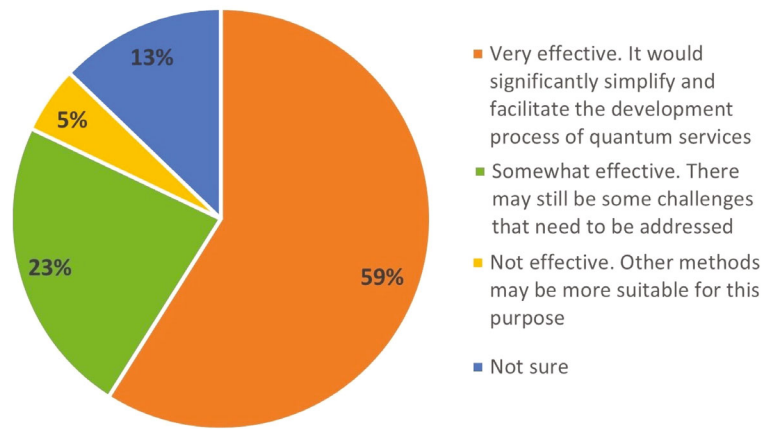


FIGURE 14 Results of RQ8.

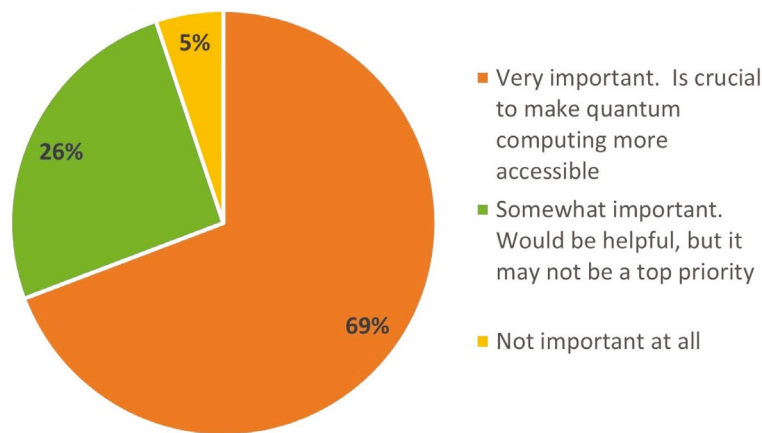


FIGURE 15 Results of RQ9.

Finally, as the results for question RQ9 reflect in Figure 15, most participants (69%) considered it very important to have a simplified workflow to generate and deploy quantum services. This indicates a strong belief that simplifying the process is crucial for making quantum computing more accessible and practical for developers. Participants recognized the challenges and complexities associated with quantum computing and believed that the proposed workflow simplification could reduce barriers and enable more efficient development and deployment of quantum services.

5 | CONCLUSIONS

This study identified several existing constraints on the creation and utilization of quantum services, highlighting the potential for adapting classical service engineering techniques and methods to the quantum realm. Consequently, developers experienced in high-level abstraction tools that streamline the development and deployment processes of classical services can expect a smoother transition to quantum service development.

To this end, we propose a method to standardize defining quantum services using the OpenAPI specification, the OpenAPI Code Generator, capable of generating the source code of quantum services from an API specification and a quantum circuit. To facilitate the automatic deployment of these quantum services, we developed a workflow for the CD of the code generated in Docker containers using the GitHub Actions tool. During this process, it checks whether the YAML format of the specification is correct, calls the code generator, encapsulates the code in a container, and finally returns the URL where the services are deployed to the user.

By leveraging GitHub Actions along with other tools such as *Q-Trans* and *Q-Deployment*, which are the contributions of this work, developers can streamline their development process by automatically generating source code for quantum services and deploying it within containerized environments.

Our proposal aims to make quantum computing more accessible by offering a tool for the automatic generation and deployment of quantum services. By automating the deployment process, our tool brings quantum computing closer to classical computing.

To validate this proposal, we generated 40 services encapsulating 20 quantum circuits. Twenty of them were implemented in Qiskit and Amazon Bracket, and another 20 were implemented using the quantum circuit composer Quirk, which is available in a public repository. We executed the pipeline with all services and successfully deployed 37 of them (92.5% success), with failed deployments being the cause of limited support for specific circuit gates.

As shown by the surveys conducted and explained in this paper, there are few tools that allow a higher level of abstraction when developing quantum software; over 70% of the respondents considered it essential to have tools that facilitate development using the architectural style proposed in this paper. In conclusion, we consider it essential to have tools that facilitate development using the proposed architectural style, which should make quantum computing more accessible to software developers by automating the deployment process.

AUTHOR CONTRIBUTIONS

José García-Alonso, Enrique Moguel and Juan M. Murillo: Conceptualization, **José García-Alonso and Juan M. Murillo:** Funding acquisition, **José García-Alonso:** Methodology, **Jaime Alvarado-Valiente and Javier Romero-Álvarez:** Investigation and experiments, **Jaime Alvarado-Valiente and Javier Romero-Álvarez:** Writing—original draft, **José García-Alonso and Enrique Moguel:** Writing—review and editing. All authors have read and agreed to the submitted version of the manuscript.

ACKNOWLEDGMENTS

This work has been partially funded by the European Union “Next GenerationEU/PRTR”, by the Ministry of Science, Innovation and Universities (projects PID2021-1240454OB-C31, TED2021-130913B-I00, and PDC2022-133465-I00). It is also supported by QSERV: Quantum Service Engineering: Development Quality, Testing and Security of Quantum Microservices project funded by the Spanish Ministry of Science and Innovation and ERDF; by the Regional Ministry of Economy, Science and Digital Agenda of the Regional Government of Extremadura (GR21133); and by European Union under the Agreement—101083667 of the Project “TECH4E-Tech4efficiencyEDIH” regarding the Call: DIGITAL-2021-EDIH-01 supported by the European Commission through the Digital Europe Program, co-funded by the Spanish Ministry of Industry, Trade and Tourism. It is also supported by the QSALUD project (EXP00135977/MIG-20201059) in the lines of action of the Center for the Development of Industrial Technology (CDTI). And by grant PRE2022-102070, funded by MCIN/AEI/10.13039/501100011033 and by FSE+.

CONFLICT OF INTEREST STATEMENT

The authors declare no conflicts of interest.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available on Zenodo at <https://doi.org/10.5281/zenodo.10477708>.

ORCID

Javier Romero-Álvarez  <https://orcid.org/0000-0002-3162-1446>

Jaime Alvarado-Valiente  <https://orcid.org/0000-0003-0140-7788>

Enrique Moguel  <https://orcid.org/0000-0002-4096-1282>

Jose Garcia-Alonso  <https://orcid.org/0000-0002-6819-0299>

Juan M. Murillo  <https://orcid.org/0000-0003-4961-4030>

REFERENCES

1. Aaronson S. The limits of quantum computers. *Sci Am*. 2008;298:62-69. <https://www.scientificamerican.com/article/the-limits-of-quantum-computers/>
2. Zhao J. Quantum software engineering: Landscapes and horizons. 2020. doi:10.48550/arXiv.2007.07047
3. Ravi GS, Smith KN, Gokhale P, Chong FT. Quantum Computing in the Cloud: Analyzing job and machine characteristics. *International Symposium on Workload Characterization (IISWC)*. IEEE; 2021:39-50. doi:10.1109/IISWC53511.2021.00015
4. Rahaman M, Islam MM. A review on progress and problems of quantum computing as a service (QaaS) in the perspective of cloud computing. *Global J Comp Sci Technol*. 2015;15(4):15-18. <https://globaljournals.org/item/5374-a-review-on-progress-and-problems-of-quantum-computing-as-a-service-qcaas-in-the-perspective-of-cloud-computing>
5. Piattini M, Serrano M, Perez-Castillo R, Petersen G, Hevia JL. Toward a quantum software engineering. *IT Prof*. 2021;23(1):62-66. doi:10.1109/MITP.2020.3019522
6. Hevia JL, Petersen G, Piattini M. QuantumPath: A quantum software development platform. *Softw Pract Experience*. 2022;52(6):1517-1530. doi:10.1002/spe.3064
7. Gonzalez C. Cloud based qc with amazon braket. *Digit Welt*. 2021;5:14-17. doi:10.1007/s42354-021-0330-z
8. Leymann F, Barzen J, Falkenthal M, Vietz D, Weder B, Wild K. Quantum in the cloud: Application potentials and research opportunities. *International Conference on Cloud Computing and Services Science*. SCITEPRESS; 2020. doi:10.5220/0009819800090024
9. Kumara I, Heuvel WJVD, Tamburri DA. QSOC: Quantum service-oriented computing. *Symposium and Summer School on Service-Oriented Computing*. Springer, Cham; 2021:52-63. doi:10.1007/978-3-030-87568-8_3
10. Papazoglou MP, Georgakopoulos D. Introduction: Service-oriented computing. *Commun ACM*. 2003;46(10):25-28. doi:10.1145/944217.944233
11. Moguel E, Rojo J, Valencia D, Berrocal J, Garcia-Alonso J, Murillo JM. Quantum service-oriented computing: current landscape and challenges. *Softw Qual J*. 2022;30(4):983-1002. doi:10.1007/S11219-022-09589-Y
12. Alvarado-Valiente J, Romero-Álvarez J, Moguel E, García-Alonso J, Murillo JM. Technological diversity of quantum computing providers: a comparative study and a proposal for API gateway integration. *Softw Qual J*. 2023;32:53-73. doi:10.1007/s11219-023-09633-5
13. Weder B, Barzen J, Leymann F, Zimmermann M. Hybrid quantum applications need two orchestrations in superposition: A software architecture perspective. *Int Conf Web Serv*. 2021; 1-13. doi:10.1109/ICWS53863.2021.00015
14. Garcia-Alonso J, Rojo J, Valencia D, Moguel E, Berrocal J, Murillo JM. Quantum software as a service through a quantum API gateway. *IEEE Internet Comput*. 2021;26(1):34-41. doi:10.1109/MIC.2021.3132688
15. Valencia D, Garcia-Alonso J, Rojo J, Moguel E, Berrocal J, Murillo JM. Hybrid classical-quantum software services systems: Exploration of the rough edges. *Int Conf Quality Inform Commun Technol*. 2021;1439:225-238. doi:10.1007/978-3-030-85347-1_17
16. Alvarado-Valiente J, Romero-Álvarez J, Garcia-Alonso J, Murillo JM. A Guide for Quantum Web Services Deployment. *Int Conf Web Eng*. 2022;13362:493-496. doi:10.1007/978-3-031-09917-5_42
17. Johnson B. Qiskit runtime, a quantum-classical execution platform for cloud-accessible quantum computers. *Bull Am Phys Soc*. 2022;2022:T28-002. <https://ui.adsabs.harvard.edu/abs/2022APS.MART28002J>
18. Lubinski T, Granade C, Anderson A, et al. Advancing hybrid quantum-classical computation with real-time execution. *Front Phys*. 2022;10:940293. doi:10.3389/fphy.2022.940293
19. Hilton J. Building the quantum workforce of the future. *Forbes Technol Council*. 2019; <https://www.forbes.com/sites/forbestechcouncil/2019/06/19/building-the-quantum-workforce-of-the-future/>
20. Jabbari R, Ali bN, Petersen K, Tanveer B. What is DevOps? A systematic mapping study on definitions and practices. *Proceedings of the Scientific Workshop Proceedings of XP2016*. ACM; 2016:1-11. doi:10.1145/2962695.2962707
21. Gheorghe-Pop ID, Tcholtchev N, Ritter T, Hauswirth M. Quantum DevOps: Towards reliable and applicable NISQ quantum computing. *IEEE Globecom Workshops*. 2020;2020:1-6. doi:10.1109/GCWkshps50303.2020.9367411
22. Weder B, Barzen J, Leymann F, Salm M, Vietz D. The quantum software lifecycle. *Proceedings of the 1st ACM SIGSOFT International Workshop on Architectures and Paradigms for Engineering Quantum Software*. ACM; 2020:2-9. doi:10.1145/3412451.3428497
23. Shahin M, Babar MA, Zhu L. Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices. *IEEE Access*. 2017;5:3909-3943. doi:10.1109/ACCESS.2017.2685629
24. Schwichtenberg S, Gerth C, Engels G. From open API to semantic specifications and code adapters. *Int Conf Web Serv*. 2017;484-491. doi:10.1109/ICWS.2017.56
25. Laso S, Berrocal J, García-Alonso J, Canal C, Manuel MJ. Human microservices: A framework for turning humans into service providers. *Softw Pract Experience*. 2021;51(9):1910-1935. doi:10.1002/spe.2976
26. Grover LK. A fast quantum mechanical algorithm for database search. *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. ACM;1996:212-219. doi:10.1145/237814.237866
27. Shor PW. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Rev*. 1999;41(2):303-332. doi:10.1137/S0097539795293172
28. Simon DR. On the power of quantum computation. *SIAM J Comput*. 1997;26(5):1474-1483. doi:10.1137/S0097539796298637
29. Farhi E, Goldstone J, Gutmann S, Zhou L. The quantum approximate optimization algorithm and the sherrington-kirkpatrick model at infinite size. *Quantum*. 2022;6:759. doi:10.22331/q-2022-07-07-759

How to cite this article: Romero-Álvarez J, Alvarado-Valiente J, Moguel E, Garcia-Alonso J, Murillo JM. Enabling continuous deployment techniques for quantum services. *Softw: Pract Exper*. 2024;54(8):1491-1515. doi: 10.1002/spe.3326

APPENDIX A. EXECUTING THE DEPLOYED QUANTUM SERVICES

This appendix describes the process of executing the deployed quantum services, which are web-based endpoints that allow users to interact with quantum computers.

Once the services containing quantum algorithms have been generated and deployed, developers can access them through the web interface, where they are hosted via the URL returned by the continuous deployment pipeline.

When developers access the web interface, they see an interface similar to that shown in Figure A1, where the endpoints of the quantum and classical services, if they have been defined in the YAML file of the specification, are located.

To execute these services, the developer must select one and click the *Try-it-Out* option. Thus, the parameters used to perform the execution can be adjusted, as shown in Figure A2.

quantum_code Everything about your quantum service code and circuits

GET	/circuit/ShorIBMquirk	Get the circuit implementation of Shor Algorithm
GET	/circuit/ShorIBMqiskit	Get the circuit implementation of Shor Algorithm
GET	/circuit/BernsteinVaziraniIBMquirk	Get the circuit implementation of Bernstein-Vazirani Algorithm
GET	/circuit/BernsteinVaziraniIBMqiskit	Get the circuit Implementation of Bernstein-Vazirani Algorithm

FIGURE A1 Examples of quantum services endpoints.

GET /circuit/GroverIBMquirk Get the circuit implementation of Grover Algorithm

Parameters Try it out

Name	Description
machine * required	quantum machine
string	
deprecated (query)	machine
shots * required	Number of shots
number	
deprecated (query)	shots

FIGURE A2 Parameters to execute a service.

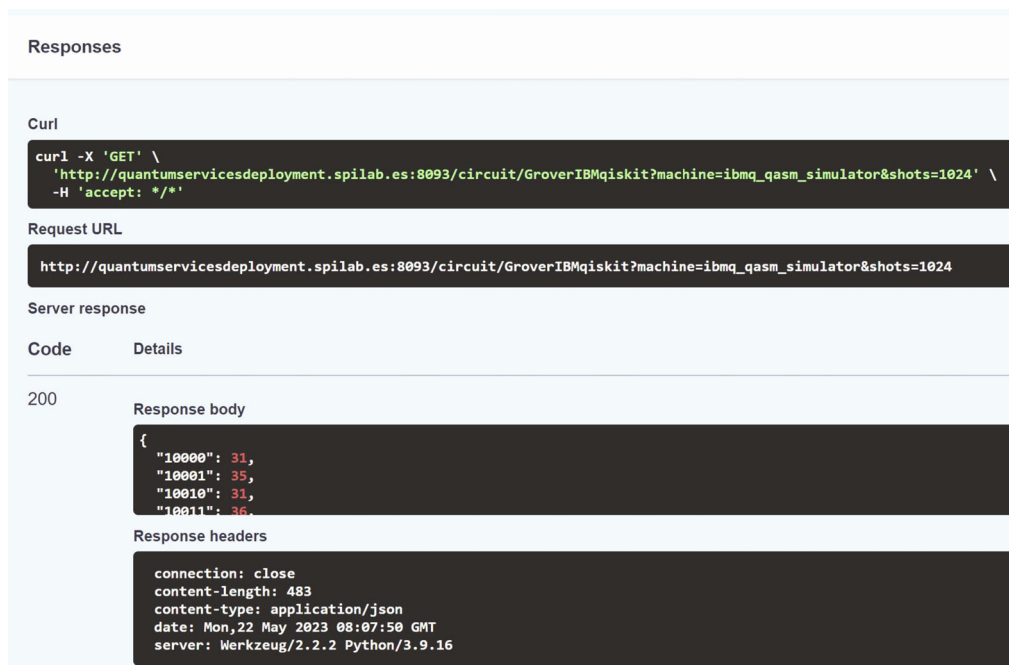


FIGURE A3 Result obtained from the execution of a service.

Specifically, in the *Parameters* section, the developer must indicate the **machine or simulator** where the execution is to be performed, and the **number of shots** indicates the number of times the circuit is to be executed on the chosen quantum machine.

The only aspect that the developer must consider is which service provider the selected service is developed for—AWS or IBM—in order to indicate the machine that corresponds to the provider.

After clicking on the *Execute* button, it begins the execution of the circuit on the selected machine and outputs a JSON with the results in the response body—Figure A3. These results represent the frequency of the occurrence of different quantum states after the execution of a circuit on a quantum machine.

Each key in the result corresponds to a binary state, and each digit represents the state of a qubit. The value associated with each key indicates the number of times a specific state is observed during the execution of the circuit. In the example, the state “10001” was observed 35 times, while the state “11101” was observed 19 times. These results provide information regarding the probability distribution of the quantum states in the experiments.

This roadmap shows how to execute the deployed quantum services; however, developers must verify the expected behavior and analyze the probability distribution obtained after execution. However, this tool saves time and effort because of its simplicity. Additionally, developers are abstracted from the low-level details of quantum hardware and programming languages. This allows them to focus on application development without the need for an in-depth knowledge of these technical aspects.

APPENDIX B. SURVEY

This appendix presents in detail the survey and the results obtained to confirm the questions raised and verify the usefulness of the pipeline presented.

B.1 Questions

The first part of the survey includes questions to obtain a profile of the participants. To do this, we posed the following general questions:

GQ1. What is your current employment situation? This question provides information to understand the situation of the participants. The options are:

- a. Student
- b. Researcher
- c. Developer
- d. Other

GQ2. What is your academic background? This question helps to understand the educational profile of the participants. The options are:

- a. Software Engineer/Developer
- b. Computer Science
- c. Physics
- d. Other

GQ3. What is your age range? The answer to this question provides an idea of the age distribution within the surveyed sample and helps to better understand the audience. The options are:

- a. 17–24
- b. 25–34
- c. 35–44
- d. 45–54
- e. 55+

GQ4. How many years of experience do you have in Software Development? This question allows us to determine the total amount of time a person has spent on software development in his or her professional career. This question helps to understand the level of experience and expertise the respondent has in that specific field. The options are:

- a. None
- b. 0–4
- c. 5–10
- d. +10

GQ5. How many years of experience do you have in quantum computing? The answer to this question provides information about respondents' prior knowledge and experience in the field of quantum computing, which may be relevant in order to assess their understanding and perspective. The options are:

- a. None
- b. 0–2
- c. 3–4
- d. 5+

Concerning the second part of the survey, research questions were asked in order to analyze the current perception of quantum software development:

RQ1. In your opinion, what is the main aspect of Quantum Computing that should be improved from a software engineering point of view? This question was raised to obtain the perspective and opinions of respondents on specific areas for improvement at the intersection of quantum computing and software engineering. It is a free text question.

RQ2. Have you designed any quantum circuits and executed them in a quantum machine or simulator before? This question aims to evaluate the level of knowledge and practical experience of the respondents in the field of quantum computing. Specifically in the design and implementation of quantum circuits. The options are:

- a. Yes, I have designed some quantum circuits and executed them in a quantum machine or simulator.
- b. Yes, but only I designed the same circuits, I did not execute them.
- c. No

RQ3. Have you developed or deployed any quantum services before? This question is related to the previous one, but in this case, the answers to this question will provide information on the familiarity of the respondents with the development of applications and services in the field of quantum computing. The options are:

- a. Yes, I have developed and deployed one or more quantum services before.
- b. No, but I have experience with quantum computing. I have worked with quantum computing before, but I have not developed or deployed any quantum services yet.
- c. No, and I have no experience with quantum computing. I am not familiar with quantum computing, and I have not developed or deployed any quantum services before.

RQ4. How challenging do you find the process of generating and deploying quantum services using current methods? This question seeks to assess the degree of complexity and difficulties perceived by respondents in the development and implementation of quantum services using existing methods seen in the course. The options are:

- a. Very challenging
- b. Somewhat challenging
- c. Not challenging at all
- d. Not sure

RQ5. Is it essential for you to have tools that will allow the development and deployment of quantum services? This question is useful to know if the respondents see the need for tools that allow in terms of facilitating and facilitating the development process of quantum services. The options are:

- a. Yes
- b. No

After introducing the basic concepts of quantum computing and showing the structure of a quantum program with current development techniques, the pipeline for the generation and deployment of quantum services by adapting techniques from the DevOps methodology for continuous software integration was presented, with real examples of use. The objective was to corroborate the ease and usability of the process presented. To this end, the following questions were asked to check its usefulness:

RQ6. Have you understood the concepts explained in the course? This question allows us to know if the contents explained during the course have been understood or not, with a view to future improvements both in the proposed process and in the course. The options are:

- a. Yes
- b. No

RQ7. How much easier do you think it would be to develop and deploy quantum services using the semi-automatic method presented compared to the current methods? This question allows us to evaluate the opinion of the respondents on the improvement degree that the presented semi-automatic approach can offer in terms of simplicity and efficiency in the development and deployment of quantum services. The options are:

- a. Much easier
- b. Somewhat easier
- c. Not much easier
- d. Not sure

RQ8. How effective do you think the Continuous Deployment (CD) approach would be in simplifying and facilitating the development process of quantum services? Responses to this question will provide information on the respondents' expectations and perceptions of the potential benefits and advantages of the Continuous Deployment approach compared to existing methods. The options are:

- a. Very effective. It would significantly simplify and facilitate the development process of quantum services.
- b. Somewhat effective. There may still be some challenges or limitations that need to be addressed.
- c. Not effective. Other methods may be more suitable for this purpose.
- d. Not sure

RQ9. How important is it for you to have a simplified workflow for generating and deploying quantum services? This question allows us to determine the importance that respondents assign to having a simplified workflow for generating and deploying quantum services. The options are:

- a. Very important. I think having a simplified workflow for generating and deploying quantum services is crucial to make quantum computing more accessible and practical for developers.
- b. Somewhat important. I think having a simplified workflow for generating and deploying quantum services would be helpful, but it may not be a top priority.
- c. Not important at all. The current process for developing and deploying quantum services may be complex, but it is necessary given the complexity and novelty of quantum computing.

B.2 Results

The results of the general questions GQ1, GQ2, GQ3, GQ4, and GQ5 are detailed below:

GQ1. What is your current employment situation? The results obtained for this question are shown in Figure 7A. We can observe that almost all the participants (85%) are currently students and the rest are mainly researchers (8%).

GQ2. What is your academic background? The results obtained for this question are shown in Figure 7B. We can see that 51% come from the computer science field, 26% are software developers and 21% come from other academic fields—such as telecommunications or web engineering among others.

GQ3. What is your age range? The vast majority are between 25 and 34 years of age (62%), and the rest of the ages are mainly between 17–24 (15%) and 35–44 (15%).

GQ4. How many years of experience do you have in Software Development? The responses to this question vary as follows: 33% of participants have 0–4 years of experience, 31% have 5–10 years of experience, and the remaining participants have over 10 years of experience.

GQ5. How many years of experience do you have in quantum computing? Most of the participants do not have any experience in quantum computing (97%). This is because it is a relatively new field of research and because of the areas in which the course has been carried out.

The following results correspond to questions RQ1, RQ2, RQ3, RQ4, and RQ5 which allow us to analyze the current perception of quantum software development:

RQ1. In your opinion, what is the main aspect of Quantum Computing that should be improved from a software engineering point of view? The general conclusion drawn from the answers is that the main aspect of quantum computing that should be improved from a software engineering perspective is the developer experience. Therefore, some key areas for improvement were highlighted by the participants, such as developing more accessible programming languages and tools for quantum programming; improving the availability and accessibility of quantum machines; standardizing access to quantum computing across different vendors to address the “vendor lock-in” problem; and increasing the level of abstraction in quantum computing. These are very useful responses as they reflect the same problems we are trying to address with our proposal.

RQ2. Have you designed any quantum circuits and executed them in a quantum machine or simulator before? The results of this question can be seen in Figure 8. Most of the participants (87%) have designed a quantum circuit but have not executed it on real machines.

RQ3. Have you developed or deployed any quantum services before? The results of this question can be seen in Figure 9. 97% of the participants have not developed or deployed any quantum services before but have worked in quantum computing before.

RQ4. How challenging do you find the process of generating and deploying quantum services using current methods? The results of this question can be seen in Figure 10. A significant percentage of respondents (38%) indicated that they find it to be somewhat challenging, while a slightly higher percentage (44%) described it as very challenging. This suggests that there are considerable difficulties associated with the current methods employed in the generation and deployment of quantum services.

RQ5. Is it essential for you to have tools that will allow the development and deployment of quantum services? The results of this question can be seen in Figure 11. Based on the results, it is evident that a majority of respondents (almost 80%) consider it essential to have tools that enable the development and deployment of quantum services. This is related to question RQ4 where also a high percentage is considered very challenging in the current process of development and deployment of quantum services.

The subsequent findings present the results pertaining to the questions that specifically assess the utility of the proposed process:

RQ6. Have you understood the concepts explained in the course? The results of this question can be seen in Figure 12. It is clear that the majority of participants (97%) have understood the concepts explained in the course. This high percentage of understanding indicates that the course content has been effectively communicated and that the proposed process is easy to understand for developers.

RQ7. How much easier do you think it would be to develop and deploy quantum services using the semi-automatic method presented compared to the current methods? The results of this question can be seen in Figure 13. Based on the responses, it can be inferred that there is a range of opinions (64%) regarding the ease of developing and deploying quantum services using semi-automatic methods compared to current methods. Additionally, 26% of the respondents believe that semi-automatic methods would not make the process “textitmuch easier” and 10% of the respondents are unsure about the impact of semi-automatic methods on the ease of development and deployment of quantum services.

RQ8. How effective do you think the Continuous Deployment approach would be in simplifying and facilitating the development process of quantum services? The results of this question can be seen in Figure 14. The majority of respondents (59%) considered a CD approach to be “textitvery effective” in significantly speeding up and facilitating the quantum services development process and 23% of the respondents consider a CD approach to be “textitsomewhat effective”, indicating that while it has advantages, there may still be challenges or limitations that need to be addressed. The remaining percentage is not sure or does not consider it effective.

RQ9. How important is it for you to have a simplified workflow for generating and deploying quantum services? The results of this question can be seen in Figure 15. A majority of participants (69%) consider it very important to have a simplified workflow for generating and deploying quantum services. (26%) of the respondents express that having a simplified workflow for generating and deploying quantum services is important. On the other hand, only 5% of the respondents consider it not important at all.

B.3 Lessons learned from the results

The proposed process for the generation and deployment of quantum services provides a more accessible approach to the development of quantum applications. This is reflected in most survey responses, which confirm the need for tools that can simplify the development and deployment of quantum services.

Furthermore, the fact that participants stated that the approach proposed in this work would make the process “*much easier*” or “*somewhat easier*” highlights the potential impact of this workflow in improving the experience of developers in quantum computing.

The following are some principles derived from the results of the questions:

- **Accessibility of Languages and Tools:** facilitate the accessibility of quantum programming languages and tools, seeking more user-friendly and less error-prone interfaces to improve the developer’s experience.
- **Integration of visual tools:** encourage the integration of intuitive visual tools for the construction of quantum circuits, facilitating the understanding and design of quantum algorithms even for those with no experience in quantum programming.
- **Abstraction in the interaction with quantum platforms:** linked to the previous principle, we find the need to create interfaces that abstract the complexity of the interaction with quantum platforms, allowing developers to focus on the logic of the algorithm without being concerned about platform-specific details.
- **Code generation automation:** promote automation in the generation of code for quantum services from specifications, simplifying the development process and ensuring consistency between specification and implementation.

- **Integration with current workflows:** from the responses obtained we also derive the need to design tools that seamlessly integrate with workflows provided by current quantum service providers, ensuring easier and more efficient adoption.
- **Development feedback:** implement continuous implementation strategies that provide rapid feedback on changes made in quantum development, accelerating the development cycle and improving efficiency.
- **Improved usability:** prioritize usability in semi-automated tools, ensuring that developers can easily adopt these methodologies and benefit from process simplification.

These principles learned from the results focus on improving accessibility, abstraction, and flexibility in the development of quantum services, looking to make quantum computing more accessible and practical for both experienced quantum computing developers and those who are just starting to dive into this paradigm.